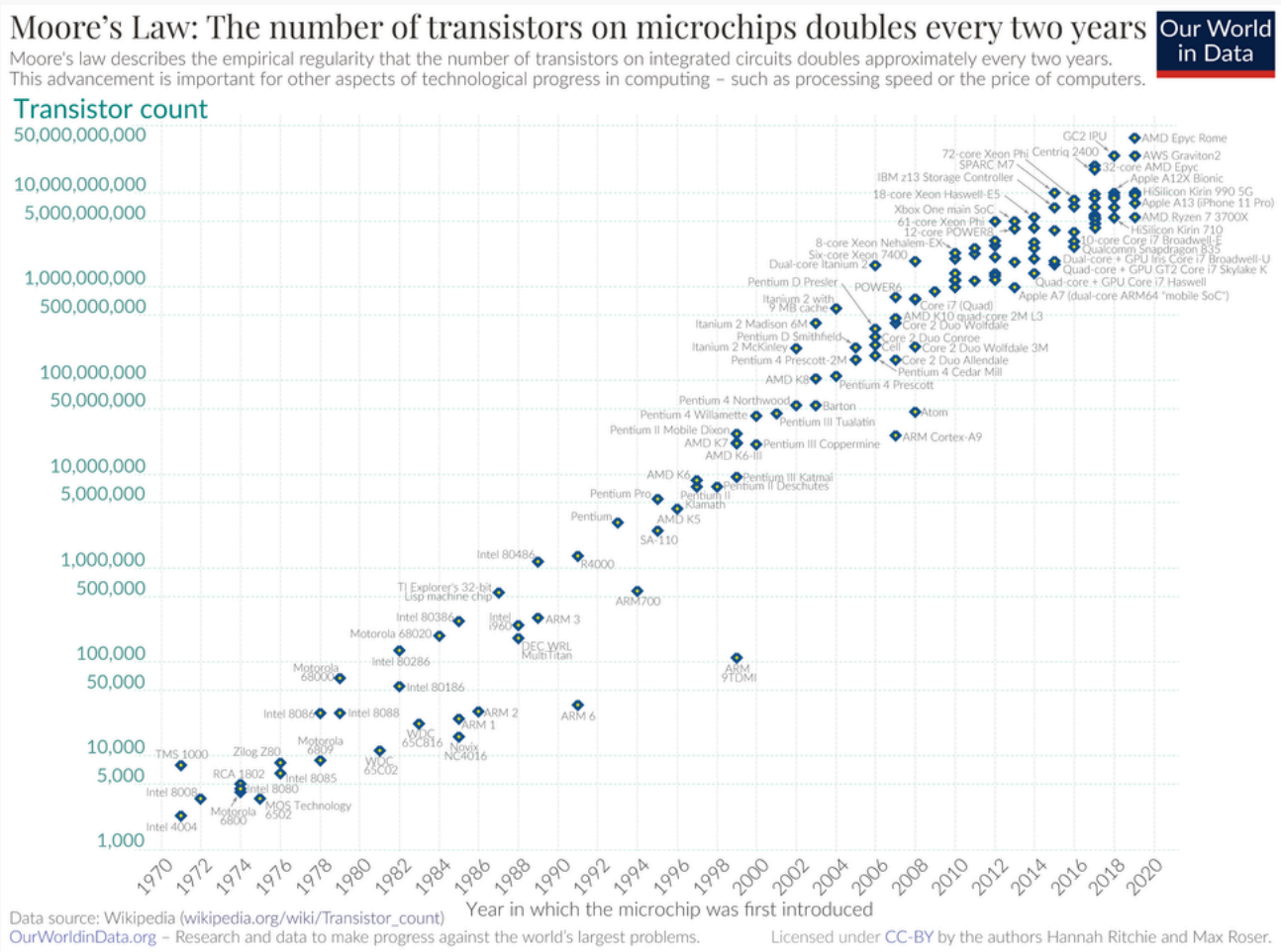
3 - Efficiency

Counting Operations

The typical modern processor operates at 4 gigahertz (GHz). The prefix "giga" means billion (10^9) and a hertz is an inverse second ($1/s$ or s^{-1}) which makes it a measure of time used to describe something in terms of the number of actions or operations per second. In other words, a 4GHz processor operates at 4 billion ($4 \cdot 10^9$) basic operations per second. 4GHz may seem extremely fast and with the historical improvement of computer speeds according to Moore's Law, it may appear that if we wait a few years processing speeds will only get faster.

Moore's Law

In the 1960's Gordon Moore, a founder of Intel, predicted based upon the historical rate of improvement of transistors in that era that the number of transistors in new computers would double roughly every two years [Moore 1965].



By Max Roser, Hannah Ritchie, CC BY 4.0

Moore's prediction held true due to the refinement of photolithography processes over decades and the resultant miniaturization of transistors that this process allowed; however, the lower bound for the scale of transistors using this processes has likely been surpassed already (around 2013) and the continued doubling of processor speeds within the same timescales is unlikely. This did not mean the end of improvements in processing speed because instead of doubling transistors chip developers began doubling the number of "cores" (roughly equivalent to a processor) allowing for multiple processes to run simultaneously. Doubling cores began in the early 2000s and processors with 24 to 32 cores are currently available to consumers. Unfortunately, a conventional single process program must be entirely re-engineered for multicore architectures in order to take advantage of the resources of a multicore system. As a result, we cannot simple count on hardware improvements to improve the efficiency of our applications.

Efficiency

What is the meaning of efficiency? It involves assessment of the utilization of a resource by a system where more efficient means less of the resource is used and it requires an objective metric and a way to measure the resource with respect to that metric. Improving or optimizing attempts to increase the efficiency of a system.

In terms of computing, we are managing two basic resources: time and space. Time in this context means computational time or the amount of processing time necessary for a program or algorithm to complete its computation and space in this context means the amount of memory used by a program.

To estimate efficiency, computer scientists build mathematical models of algorithms and their informed understanding of the system to model both the utilization of memory for a program and the amount of time necessary for the program to fully execute. A mathematical function can abstractly and generally describe the efficiency of an algorithm which means a precise mathematical function is not necessary. In other words, computer scientists classify an algorithm's performance based on the largest mathematical term that approximates its efficiency.

Efficiency in Space

Space in this context means primary memory or RAM. In general, every variable you allocate must exist somewhere in primary memory during the run-time of a program. Efficiency in terms of space means how much memory an algorithm requires to execute. Fortunately, the amount of primary memory available in personal computers has grown according to Moore's Law over the last 50 years, so the modern computer has massive amounts of memory compared to its counterpart in the late 1970's.

The size of primary memory in modern systems today is more than 16,000x times larger than it was in the late 1970’s. In 1995 the cost of 1MB of memory was $50; today 16GB of memory can be bought for about $50, and 16 GB is about 16,000x larger than 1MB.

With so much memory available, developers often overlook space efficiency especially in terms of applications running on personal computers; however, there are still a significant number of platforms where space efficiency is non-trivial and these are typically platforms with very small form factors. Worrying about space has had a lower emphasis in modern computing, but we need to still need consider memory needs especially in emerging computing domains like wearables.

Throughout this course, we will look at the memory organization associated with Java code which will give us a view into the space costs of a program and we will realize a metric that we can use to generalize the amount of space used by a program; however, this will be a diminished focus in this class compared to efficiency in terms of time.

This material will generally summarize space complexity anticipating that the reader will further investigate analysis on their own and will occasionally discuss space complexity at depth for clarity. Courses primarily focus on time complexity because there is no easy remedy for computational cost while the principles of space complexity are very easy to infer from time complexity, so our focus is typically on the more important one (time) with the assumption that students will ultimately recognize the other one (space).

Efficiency in Time

Moore's law held in terms of processing time until mid 2000's but processors have not generally grown much beyond 4GHz since around 2004 and the processor speeds are unlikely to improve more than 2-4x beyond what we have today without a significant breakthrough in physics. What are the physical limits on processing speed and why are processor speeds unlikely to continue to grow according to Moore's Law?

A wire can only be so small to reliably carry a signal. The wires in a processor are on the order of nanometers and it is difficult to make them any smaller while still being reliable. Reducing the scale of wires has increased the number of wires we can pack into the same area of a processor. As the number of wires increases, so too does the power requirement for the system increases and with an increase in the amount of power comes a corresponding increase in waste heat. Heat, specifically getting rid of it, is harder to manage as the size of wires gets smaller and more components are crammed onto the same size area in a processor. There is a difficult balance between wire size, power needs, and waste heat that bounds the minimum size of a conventional processor; however, a significant advance in physics or materials might allow for more miniaturization and corresponding processor speed increases, but these advances are not on the immediate horizon. However, it is one of the reasons industry continues to research semiconductors.

If processors have not been getting faster, how have computers been getting better since the 2000s. In other words, what was the big architectural change in processors that occurred in the 2000's.

Multi-Core Processors

The first dual-core processor was released in 2001 [IBM Power4]. The underlying principal is that two or more processing units can be packaged into a single processor allowing for two processes to compute in parallel. There is a distinction to be made here between multiple processor and multiple compute units which endures today and that is that a multi-core processors processing units are not independent processors. Two independent processors in a computer is called a multi-processor system while a system with a single processor but with multiple computing units inside the processor is called a multi-core system. The distinction is important because a multiple processor system can run multiple, completely independent programs (the number is dependent on the number of processors) simultaneously while a multi-core system allows multiple sub-processes belonging to the same program to be run simultaneously. From an outside the application perspective, the multi-core system still acts like a single processor running a single program, but from an inside the application perspective, a program may be broken into different sub-processes that perform different roles. To further make a distinction in the nomenclature between multi-process and multi-core systems we call the sub-processes in a multi-core system threads instead of processes to indicate that they are not fully independent processes.

This advancement allowed chip manufactures to continue to claim that they were doubling processing speed according to Moore's Law even without substantially increasing the processing speed of the processing unit by doubling the number of cores in a processor roughly every two years. It is not uncommon to buy a processor with 8 to 16 cores today in the average laptop and some very high-end desktops are already shipping with 32 cores. However, the multi-core processing model does not actually double processing speeds unless a program is specifically designed and well constructed for a multi-core environment. The software designed in an introductory level computer science class is not designed for a multi-core system, so these programs are effectively capped to run at speeds associated with a computer designed in 2000.

The Graphics Revolution

The insights of parallel processing also resulted in a massive redevelopment of computer graphics technology. Computer graphics had required more and more specialized processing to produce the computer graphics of the late 1990's and early 2000's; however, specialization was also limiting the future of computer graphics. Computer graphics cards of this era were highly engineered and optimized to perform a very specific sequence of subtasks in order to perform one specific task, generating computer graphics. This was an incredibly specific process necessary to sustain the growth of the still somewhat narrow computer graphics card market. NVIDIA dominated the computer graphics market by the end of the 1990's and began to look at other products in the market to continue its growth via gaming support.

In the late 1990's and early 2000's, other computational capabilities were being recognized in the market, such as improving the physics of computer simulations. Companies went so far as to produce expansion cards and libraries to calculate physics for games or other simulations [NovodeX AG] and NVIDIA acquired much of this intellectual property because it was a natural extension of their market. NVIDIA faced an important decision as a result of these acquisitions: it could continue to build and maintain an extremely specialized, expensive, and limited graphics pipeline or it could reinvent the process by generalizing graphics as a computational linear algebra equation and designing a card on this new concept.

In 2009, NVIDIA released a completely new technology for computer graphics processing using massive parallel processing, i.e. NVIDIA’s CUDA model, to improve performance. This processing model uses hundreds of very specialized processing units to compute the mathematics underlying computer graphics. What NVIDIA recognized is, in the general sense, that computer graphics is essentially a set of linear algebra or mathematical equations. The physics for more realistic physical behaviors is also a similar set of equations. This means that NVIDIA simplified the process of graphics by creating cards that are very efficient at calculating large sets of equations simultaneously and added the capability of supporting physics simply by generalizing the cards.

NVIDIA introduced CUDA in 2009 and it took about a half a decade to refine the tools to expand accessibility beyond game developers and scientists; however, other applications quickly emerged where similar large mathematical problems are at the heart of the problem, namely crypto-mining and artificial intelligence.

Computer science has been studying and developing AI theory for more than seventy years but the computational needs far exceeded the computational capacity of systems before CUDA entered the market. Developers have only recently been able to actually apply the theory of AI and take advantage of all of the years of research and development into the algorithms now that there is sufficient hardware to support it, hence the AI boom of the early 2020's.

Regardless of these computational advancements, we continue to be more concerned about the amount of time it takes for a program to execute. Why? Because it is trivial to write a program that cannot be solved even on a modern computer if the algorithm is naively designed and the input is sufficiently large. We can't always solve a complex problem by just waiting around for the technology to improve, we do have to plan and design our models in such a way that they are computationally solvable on current technology or we have to shelve that model until it is computationally feasible.

In light of this important objective, we need a measured way to evaluate efficiency objectively so that we can make decisions guided by reasoned, scientific principles and not by hunches and guesswork.

How to measure efficiency?

At first consideration, it may seem reasonable to measure the run time of an application as a measure of the application's computational efficiency; however, run-time measurements have very limited use and have significant, hidden built-in cost. Consider the following run-time data measured in seconds gathered from a variety of different platforms with applications using different data structures:

Operation	2006 Mac-OSX	SUN Server Unix	2005 Win-XP
Insert (linked list)	3.412	2.969	3.562
Insert (array list)	3.39	1.136	1.406
Search (linked list)	0.626	4.675	0.687
Search (array list)	0.629	4.177	0.469
Get (linked list)	0.432	3.479	0.719
Get (array list)	0.0	0.002	0.0

http://www2.seas.gwu.edu/~jrt/cs1112/modules/module7/module7.html

While the above may initially seem reasonable consider that it tells us nothing about how these algorithms will perform given different data set, and without the data used for these tests, the tests themselves lose all context. In other words, it does not address potential variations in performance subject to variations in the size of the input, so there is no predictive capacity with regard to input in these measurements. It would be better if the analysis abstracts away the data so that the analysis can be performed without specific data yet takes into account the impact of data on the algorithm's performance.

The cost of measuring these algorithms is also significant and difficult to capture. Consider that these algorithms could not be evaluated until they were completely implemented meaning that the analysis, design, implementation, testing, and debugging time for each algorithm has already been absorbed before it was possible to measure their performance. As a results, the developmental investment is at least double (two data structures are being compared) yet we may only use one of the data structures in the final product. Even if we need both data structures, we could not make any predictions about performance before completing the development, so all other development depending on these data structures must be on hold until this phase of development is

complete. A much more cost effective approach would allow measurement of the efficiency of these algorithms before any implementation begins.

Computational Efficiency

Variations in computational performance exist because there are a significant differences at many levels in the computational model surrounding the average application. It is quite rare for two computers to contain the same hardware and hardware manufactures optimize different aspects of their products, so performance will naturally vary when the same application is run on two different hardware configurations. On top of the hardware, there are significant differences in operating systems (OS) design and implementation, such that even if the same hardware is used for two different OS's, performance will naturally vary when the same application is run on two different operating systems. These same differences between language implementations also exist which contributing additional variation. Additionally, the number and nature of programs running on a machine at a given time also affect the run-time of a program. The run-time measurement of program is affected by a large number of outside variables that are difficult to capture, so while while run-time has some level of objectivity, the variables contributing performance differences are hard to control for and are difficult to isolate as contributing factors.

Remember that we want to measure the efficiency before even implementing the algorithm. In other words, we want to use the power of prediction, the key capability of science, via well tested models to approximate the computational efficiency of an algorithm before investing implementation, testing, and debugging resources and time into the development cycle. We need to know at concept time whether our algorithm is efficient enough or whether we need to investigate alternative algorithms. In other words, is there a model that can estimate the computation efficiency of an algorithm without using real-time measures? Additionally, a good model will abstract away hardware, operating system, and language specifics and will focus instead on the central property that really determines the performance of an algorithm, namely the size of the input? In other words, is there an abstract but objective measure that allows us to predict and compare the efficiency of algorithms based on the size of the input?

A Simple Computer

We can design a simple computer that only directly supports a few specific mathematical operations yet is still capable of performing a wide variety of math. For example, if our simple computer supports four basic operations, `store`, `add`, `negate`, and `loop`, it can perform addition, subtraction, multiply, division, exponentials, and a host of other mathematical operations.

The primitive computer has four basic operations:
1. Store/Assign an integer value to a named location in memory, ex. `X = 0`, is expressed using the binary `=` operator where the variable on the left of the operator is assigned the value of the expression on the right of the operator.
2. Add two integer operands is expressed using the binary `+` operator, ex. `A + B`. Given two operands, `A` and `B`, the add operation `+` returns the sum of the two operands.

The primitive computer has four basic operations:
3. Negate one integer operand, `A`. That is, given `A`, the negate operation returns `-A` and is expressed using the unary `-` operator. If the input `A` is positive the result will be negative and if the input `A` is negative the result will be positive.
4. Loop using a basic conditional check. The condition is checked at the top of the loop. If the condition is true when the top of the loop is reached, the process will repeat the contents of the loop. If the condition is false when the top of the loop is reached, the process will skip the contents of the loop. The syntax necessary is roughly equivalent to the Java `while` loop syntax.

From this set of simple operations, we can define a subtraction operation, `sub(A,B)`, because subtraction is a form of addition, `A + (-B)`. We can also define a multiplication operation, `mul(A,B)`, because multiplication is repeated addition, `A+A+...+A` where the number of `A`'s added is equal to `B`. From these basic concepts and mathematical axioms, we can build a rather capable computer; however, it raises the question of how efficient will each operation be.

Consider the following definitions:

```
add(A,B):
    return A + B

sub(A,B):
    return A + (-B)

mul(A,B):
    result = 0
    i = 0
    while i < B
        result = result + 1
        i = i + 1
    return result
```

Is it possible to quantify the efficiency of `add(A,B)` in terms of the number of operations with respect to the size of the input. In other words, does the number of operations in `add(A,B)` vary in any way given inputs of `add(1,2)` and `add(100,100)`. The answer is no because in both cases, only a single line is operated on regardless of the size of the input values.

Conversely, consider whether or not `mul(A,B)` is more or less efficient than `sub(A,B)`? While `sub(A,B)` consistently requires two operations on the primitive computer regardless of any outside variables, `mul(A,B)` is not so straightforward. In fact, the efficiency of `mul(A,B)`, at least in terms of the algorithm presented above, varies with respect to the size of `B`, and in general, it varies with the size of the operation's input. The follow-up question is how much does `sub(A,B)` and `mul(A,B)` vary with respect to the size of the input so that we can compare these two algorithms.

Subtract never varies in the number of operations. In fact, subtract only required two operations any time it is called; therefore the number of operations never vary with respect to the size of the input. As a result, `sub(A,B)` can be described as having a constant number of operations.

In contrast, multiply is dependent on the size of the input values. Why, because multiply involves iteration, in this form via a loop. While the number of operations in the multiply loop is constant, the number of times the loop operates depends on the input. For example, if `A=10` and `B=10`, the loop would iterate 10 times because it is dependent on the size of `B` based on the implementation above. However, if `A=1000` and `B=1000`, the loop would iterate 1000 times because it is again dependent on the size of `B`. The most important observation here is both that the number operations varies with the size of the input and the mathematical relationship that describes the relationship.

In terms of the example, the relationship is linear between the size of the input and the number of operations. For other algorithms, this relationship may be defined by another mathematical function. Given an appropriate, comprehensive analysis, we do not even need to run the program to measure this, and in fact we can measure this efficiency from a diagram or pseudo-code. Most importantly, a programmer can predict the performance of an algorithm before implementing it.

Complexity Analysis

Our goal is to define a measure for **Algorithmic Complexity** and a basis for **Complexity Analysis**. We can use the same measure to quantify an algorithm's performance in terms of its processing time, *i.e.* **Time Complexity**, and memory usage, *i.e.* **Space Complexity**. In other words, we will need an objective measure for "how much the computation time of a program scales with respect to the size of the input," and it also is a way to measure "how much our program's memory needs scale with input". Note that our measure, critically, is always defined in terms of how many elements exist within the input set.

This metric is predictive, generalized, simple, and universal which allows us to compare the complexity of any two algorithms and make reasoned arguments based on the objective, clear relationships we can infer from the mathematical nature of quantitative analysis. Additionally, as a predictive measure, we can now predict the efficiency of an algorithm from a rough outline of the operations which means we do not need to invest development time before we can determine whether a candidate algorithm is a good fit for our program requirements. The metric is generalized, so we can compare the efficiency of two algorithms regardless of the architecture, operating system, language, or other applications that they may compete with or on. Because our measure is sufficiently generalized, we can compare completely unrelated algorithms. The simple, abstract measure ensures an algorithm's complexity is reduced to a single, easily communicable, easily interpretable measure such that any computer scientist will understand the same model which makes it universal, meaningful, and useful. It is also quantitative which allows us to clearly differentiate and rank algorithmic performance.

For every time complexity analysis, it is necessary to <u>always</u> address three components:

1. Must always define variables used in the analysis
2. Must make a claim/hypothesis in terms of Big O
3. Must defend that hypothesis with an analysis

In other words, an analysis is incomplete and generally meaningless without defining variables, defining a hypothesis, and defending the hypothesis. Most students feel most comfortable with the above order and argument structure; however, a reasonable alternative is:

1. Define variables used in the analysis
2. Perform analysis
3. Summarize as Big O

What is meant by defining variables? We will generally always express complexity analysis in terms of n and we generically define n to be "n where n is the size of the input". But what does this mean?

In many cases we are dealing with only a set of size n like an array, so n is an abstraction representing the number of elements in the array. You may define n to be "n where n is the size of the array" or "n where n is the number of elements in the set", but the generic definition always works. Some algorithms may have multiple or alternate inputs that are functions of n which affect the time complexity and we will generally try to reduce these cases down to be in terms of n to eliminate all other variables. Ultimately, you will define n as a set that encompasses all inputs, but it is reasonable for now to narrowly consider it as the largest variable input set like an array that is passed in or the tree to be operated on.

There is no single correct answer here on how to perform an analysis and we will spend much of the course using a variety of approaches ranging from raw math to diagrammatic approaches to pseudo-code in order to perform analysis. The key takeaway is that this is an absolutely necessary step in both student and program development and it is the primary skill we are developing in this course. Without performing analysis and supporting your argument you have not fulfilled the programming task. This course is designed to force regular exercise on time complexity and this skill set will be assessed on virtually every meaningful evaluation.

Big O

While analyses may take on a variety of approaches and forms, efficiency claims are reduced to a single measure, Big O Notation. Big O Notation follows the form – O(). For example, these are all valid Big O arguments:

Polynomial (P) Time	$O(c)$ or $O(1)$ $O(\log_2 n)$ $O(n)$ $O(n \log_2 n)$ $O(n^2)$ $O(n^3)$ $O(n^4)$	Constant Time Logarithmic Time Linear Time Log-linear Time Quadratic Time Cubic Time Quartic Time
Non-Polynomial (NP) Time	$O(2^n)$ $O(n!)$	Exponential Time Factorial Time

Of course there are more. Any mathematical function can be expressed in Big O notation, but there are some that are more common than others. Regardless, there are also some important, common features for Big O notation.

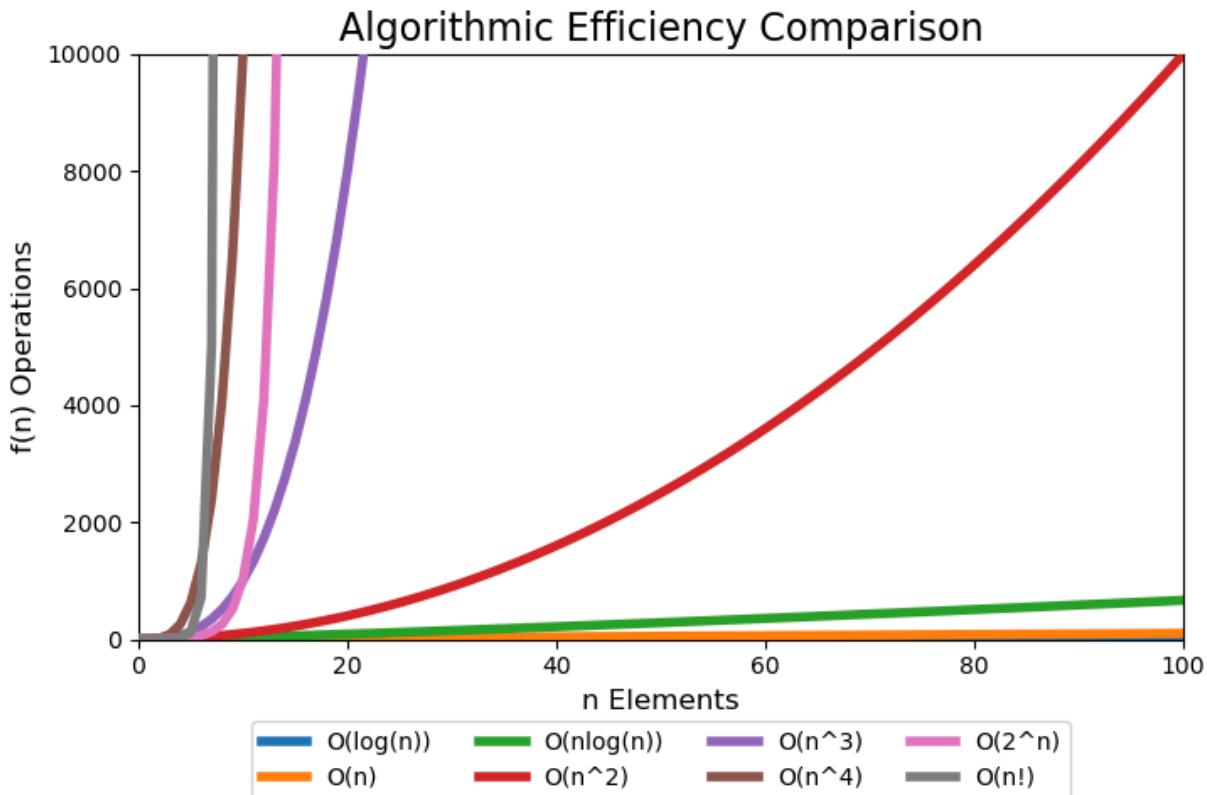
Big O allows us to drop all term coefficients. For example if we were to estimate a function with the following function $f(n) = an^2$ we would drop the coefficient a and generalize the function in Big O as $O(n^2)$. The reason behind dropping coefficients is that it can be difficult to accurately calculate coefficients from code and for large inputs the variable term itself dominates the equation. In other words, for really large numbers of n, the difference between $f(n) = 2n$ and $g(n) = 3n$ is much less relevant than the difference between how the order of the largest variable term, such as comparing $f(n)$ with respect to $h(n) = n^2$, would affect overall efficiency. On the scale of large numbers $f(n)$ and $g(n)$ take essentially the same amount of time while $h(n)$ will take considerably longer.

Big O also focuses only on the largest term and allows smaller terms to be dropped. For example if we were to estimate a function with the following function $f(n) = n^3 + n^2 + n$, we would drop both the n^2 and n terms and generalize the function in Big O as $O(n^3)$. We do this because the growth of the function will be dominated by the largest term and the smaller terms have a relatively negligible effect on the growth of the function for large inputs.

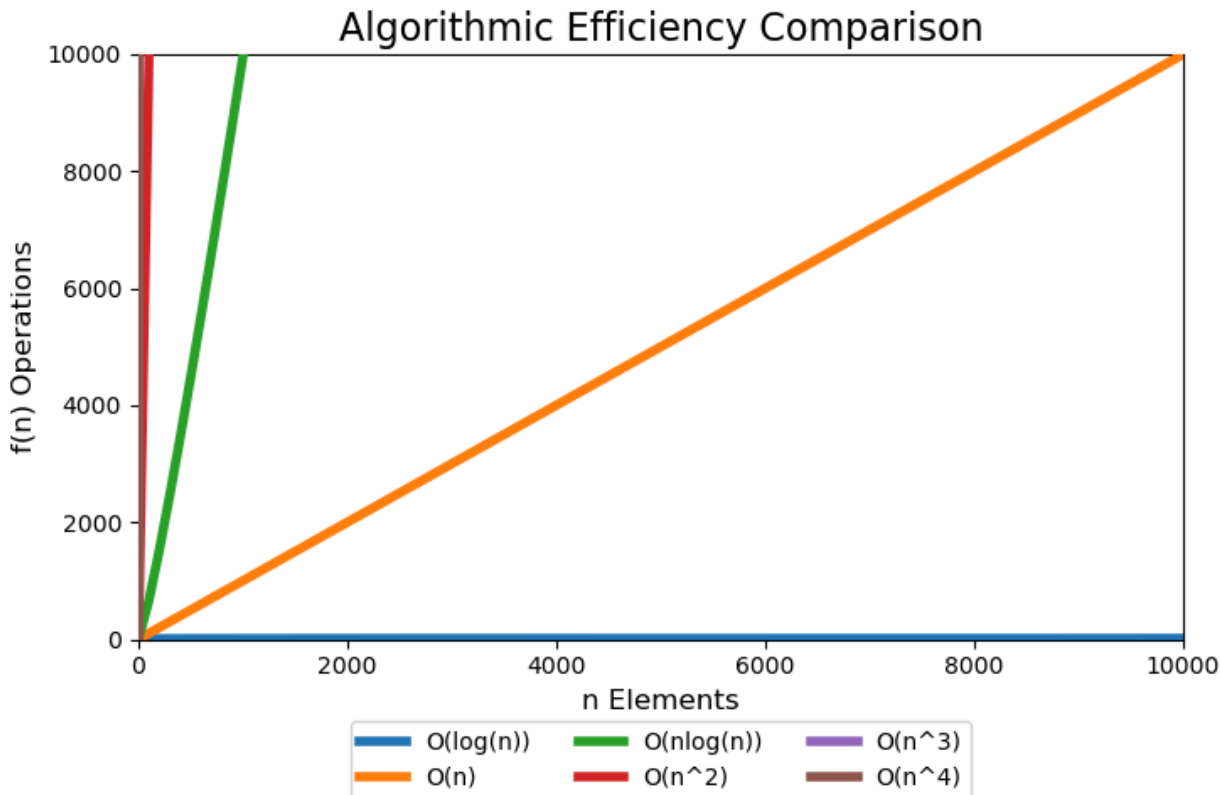
There are other forms of similar analyses such as Big Omega and Big Theta, but it is most important to introduce Big O to students in this course.

For a small input, most polynomial algorithms will solve in a reasonable amount of time, however, non-polynomial algorithms with even modest sized inputs may not solve. The following plots are limited in the scale that can be shown, but they endeavor to give an idea of the differences between algorithms of different Big O classifications. Remember that in terms of efficiency, having a small y-value is better or more efficient. This translates to any line below another line is a more efficient algorithm.

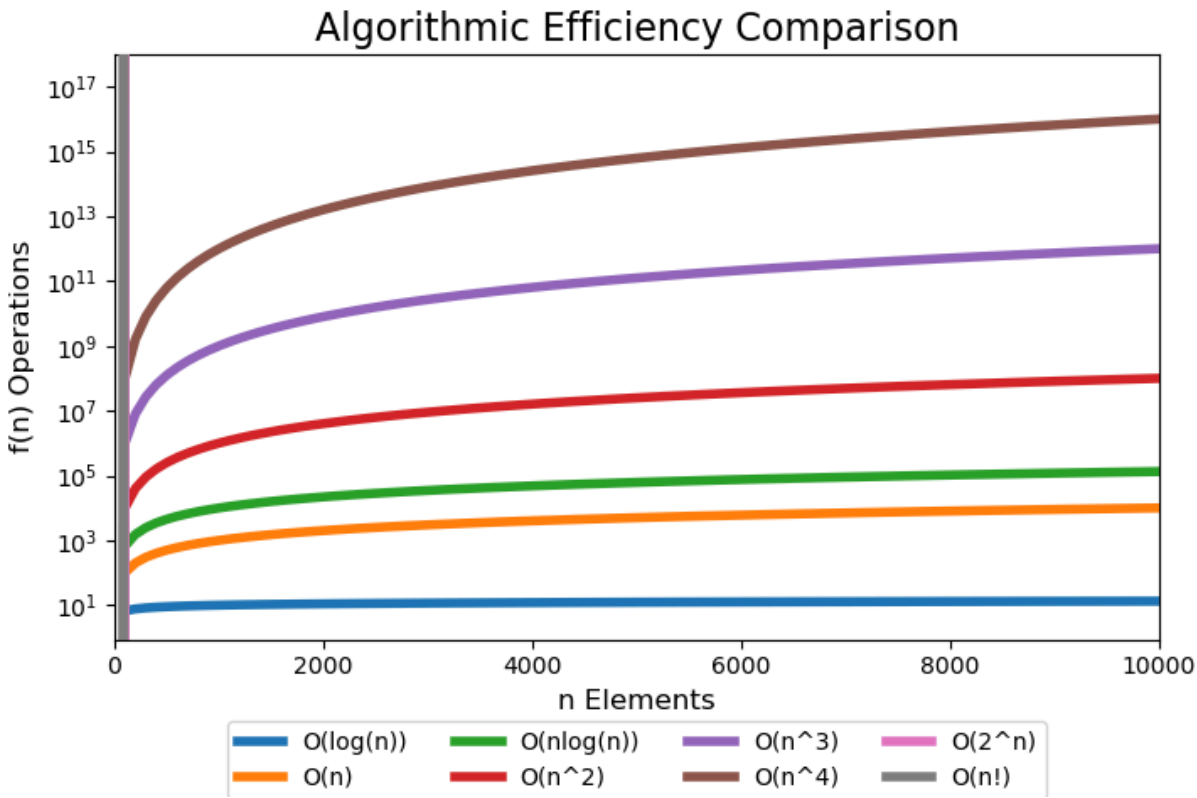
The first plot below illustrates the performance of functions for small inputs. Note that the quartic $O(n^4)$ algorithm is "above" or "to the left" of the exponential $O(2^n)$ on this plot; however, once they cross at around $n = 15$, the quartic algorithm will always be below the exponential over the rest of the domain. This same characteristic will arise for all polynomials cubic n^3 or greater. For example, the cubic and exponential cross with the purple line $O(n^3)$ crossing the pink line $O(2^n)$ and a rapid divergence after crossing. However, this plot is far from comprehensive and may be a bit misleading because the scale of x is relatively small.



The second plot attempts to illustrate the functions for what may be considered an extremely small set of 10000 elements. Note that it is difficult to discern much other than logarithmic is extremely efficient while everything "above" $O(n^2)$ is hard to interpret.



This third plot addresses the weaknesses of the second plot by using a logarithmic y-axis. Note that the growth of y-axis is not constant but each step is an order of magnitude greater which compresses the data in the upward y scale and allows for extremely large numbers to be represented. One artifact of this is that all of the polynomial functions have a pronounced turn on the plot and appear to flatten which could falsely be interpreted as an asymptotic behavior. What is evident from the plot is that the non-polynomials even for very small values of n are unsolvable because they are "off the chart" and that there is a clear hierarchy of polynomials indicating which algorithms will perform better than others according to their Big O classification.



Big O will be the most important concept of this course and it will be a recurring theme as a result. It will feature prominently in every discussion involving algorithms.

Other Complexity Analyses

The field of algorithm quantitative analysis has more measures beyond just Big O. Two similar measures are Big Theta, Θ, and Bit Omega, Ω, which are expressed in the same ways as Big O using similar analytical techniques but with different analytical envelopes. Big O is strictly limited to a general worst-case envelope that every function shares which is what makes it applicable to all algorithms while these others are applicable to more specialized cases such as Big Theta only operating on algorithms where the best-case and worst-case match. In other words, these measures can be more nuanced and actually give better estimates of the performance for specific classifications of algorithms.

Above, we declared that the material will focus more on time complexity at the cost of some discussion of space complexity for the sake of teaching the most relevant material, and so here we too will narrow our focus on time complexity to Big O in the remainder of the materials for the same reasons. We memorialize these alternative measures here to make the student aware that there are other more sophisticated measures of algorithmic time complexity, but in this course, we will perform algorithm time complexity analysis exclusively using Big O evaluations and will leave further definition, instruction, and discussion of these complexity analyses to the domain of a discrete structures or a strictly focused algorithm course.

We do encourage students to further explore Big Theta and Big Omega analysis, but they will remain beyond the scope of this course as we will wish to focus on one single metric so that there is no confusion among students at this level and because Big O analysis is generally all the industry demands. We wish to encourage students to explore Big Theta and Big Omega more as having these skills can distinguish you from the rest of the labor market

or better equip you for analytical/optimization work in programming, but this will be the extent of addressing the metrics in this material for the aforementioned reasons.

Review

1. What variables affect the execution time of a program?
2. Why do we use algorithmic time complexity to measure a program's efficiency rather than execution time?
3. What are the benefits of using an abstract measure like algorithmic time complexity over a rigidly specific measure like execution time?
4. Why can we ignore coefficients and smaller terms in an algorithmic time complexity analysis?
5. What is Big O notation?
6. Why does the growth rate of Big O meaningfully capture the efficiency of a program over its entire input?
7. Why do we generalize time complexity analysis to an evaluation in terms of "n where n is the size of the input"?
8. Why does time complexity analysis capture information about arbitrarily large data sets that cannot be captured by measurement of execution time?
9. Why does time complexity analysis allow us to measure efficiency of pseudo-code?
10. How is algorithmic time complexity a predictive measurement?
11. Why is any real-time measure a reactive measurement?
12. Why is it much more efficient in the design space to use algorithmic time complexity rather than real-time measurement?
13. Why would a real-time measurement be almost useless if the size of the input was significantly changed.
14. Why are the efficiencies generally grouped into polynomial (P) time and non-polynomial (NP) time?
15. In the plots, an input of $n = 10000$ is claimed to be small. What would be considered large? Justify.
16. The chapter suggest that the same approaches can be used for space complexity. Why is space complexity of diminished concern for the average application? What types of applications would require optimization of space complexity and why?